

ExpenseTracker – Final Project Report

A Full-Stack MERN Personal Finance Tracker



Project Title	ExpenseTracker – Personal Expense & Budget Management System
Submitted By	Sharma Saurabh
Roll Number	250863131007
Institute	Laxmi Institute of Technology (LIT), Sarigam – Laxmi Vidyapeeth
Department	Computer Engineering (CSE)
University	Gujarat Technological University (GTU)
Project Guide	X
Academic Year	2025-2026
Report Date	08 October 2026

1. Introduction

ExpenseTracker is a complete personal finance management web application built on the **MERN stack** (MongoDB, Express.js, React.js, Node.js). It allows a user to record income and expenses, organise them into categories, set monthly budgets, track savings goals, automate recurring payments, and understand spending behaviour through interactive analytics and monthly reports.

The project is a ground-up rebuild: the database design, backend API, frontend UI, and design system were all written fresh, while keeping every feature of the original reference implementation and adding a substantial set of new capabilities – including a **light/dark mode**, a **multi-theme design engine** (8 selectable themes), **CSV export**, **favorite transactions**, **receipt uploads**, **smart notifications**, **month-over-month reporting**, and a much stronger **validation and security layer**.

Design language: **Option 8 – Warm White + Espresso + Sage**, rendered entirely in the **JetBrains Mono** typeface for a clean, engineering-first, monospaced look.

2. Objectives

1. Build a full-stack CRUD application covering real personal-finance workflows (transactions → budgets → goals → reports).
 2. Implement every original feature (auth, transactions, categories, budgets, savings goals, recurring expenses, analytics) without regressions.
 3. Add modern product features: dark mode, selectable themes, CSV export, favorites, receipt attachments, smart notifications, richer reports.
 4. Apply a distinctive, consistent design system (Option 8 palette) with micro-interactions and polished empty/loading/error states.
 5. Keep the code **simple and easy to explain** in a viva: sequential `await` (no `Promise.all`), small single-purpose files, no Docker, no third-party APIs.
 6. Follow security best practices: JWT auth, bcrypt hashing, server-side validation, centralized environment configuration.
-

3. Tech Stack

Layer	Technology	Why chosen
Database	MongoDB (local, DB <code>expenseTracker</code>)	Flexible schema for varied finance records
Backend	Node.js + Express.js	Lightweight REST API, easy to explain
ODM	Mongoose 8	Schema validation, populated references
Frontend	React 18 (Vite)	Fast dev server, component model
State	Redux Toolkit	Predictable, minimal boilerplate slices
Routing	React Router v6	Declarative protected routes
Charts	Recharts	React-friendly bar/line/pie charts
Styling	Tailwind CSS (<code>darkMode: class</code>) + custom component classes	Rapid, consistent styling
Typography	JetBrains Mono (full UI)	Distinctive monospace identity
HTTP	Axios (centralized <code>services/api.js</code>)	Single interceptor for auth + errors
Toasts	<code>react-hot-toast</code>	Lightweight feedback
Uploads	Multer (2 MB receipt images)	Server-side receipt storage
Auth	JWT + <code>bcryptjs</code>	Stateless auth, hashed passwords
Security	<code>helmet</code> , CORS allow-list	Hardened defaults

Ports: Backend `5000` , Frontend `5173` (Vite proxies `/api` and `/uploads` → backend, so no CORS pain in dev).

4. Design System

4.1 Option 8 – Warm White + Espresso + Sage

Token	Hex	Usage
Background	#FAF8F3	Page background
Text	#29231F	Primary text
Primary (Espresso)	#493B32	Buttons, active states, sidebar brand
Accent (Sage)	#78816A	Charts, links, highlights
Sage tint	#E9EDE4	Sidebar, soft panels
Beige tint	#E9DFD1	Stat cards, hero balance card
Card	#FFFFFF	Surfaces

All colors are exposed as **CSS variables** (`--bg`, `--text`, `--card`, `--border`, `--primary`, `--accent`, `--sage`, `--beige`, ...) defined for both `:root` and `.dark`, so switching theme or dark mode is a pure CSS-variable swap – no component needs to know about colors.

4.2 Theme Engine (New Feature)

- 8 independent theme files in `client/src/themes/` (`option1...option8`), one file per theme, plus a small registry (`themes/index.js`) exposing `getTheme()` / `applyTheme()`.
- User picks a theme on the **Profile → Appearance** section; selection persists in `localStorage` (`theme`, `themeId`).
- `dark` toggle is independent and also persisted; navbar moon icon toggles it from any page.
- Screenshot `14-theme-burgundy.png` proves one click swaps the whole identity of the app.

4.3 Interaction & Motion

- All transitions **150-250 ms**, pure CSS (no JS animation libraries): button hover, card hover-lift, modal fade+scale, progress-bar fills, skeleton shimmer, toast slides, flash-highlight on newly added rows.
- Accessibility: visible `:focus-visible` outlines, keyboard-dismissable modals (Escape), labelled inputs.
- Number inputs: `inputMode="decimal"`, sanitized (digits + one dot + max 2 decimals), browser spinners hidden.

5. Architecture & Project Structure

```
ExpenseTracker/
├── server/                                # Express API
│   ├── src/
│   │   ├── config/env.js                # ONLY file that reads process.env
│   │   ├── models/                      # 7 Mongoose models
│   │   ├── middleware/                  # auth (JWT), error, upload (multer)
│   │   ├── utils/                      # notify, processRecurring, buildCsv, sanitize...
│   │   ├── controllers/                 # 8 controllers
│   │   ├── routes/                     # 8 routers (40 endpoints)
│   │   ├── seed/seed.js                 # 2 demo users + data
│   │   ├── app.js server.js
│   │   └── .env / .env.example
│   └── client/                          # React (Vite) SPA
│       ├── src/
│       │   ├── config/env.js            # ONLY file that reads import.meta.env
│       │   ├── themes/                  # 8 theme files + registry
│       │   ├── features/                # 8 Redux slices + store
│       │   ├── services/api.js          # axios instance + interceptors
│       │   ├── layout/                  # Sidebar, Navbar, MainLayout
│       │   ├── common/                  # Modal, EmptyState, Skeletons, charts
│       │   ├── pages/                   # 11 pages
│       │   ├── hooks/ utils/
│       │   └── index.css                 # variables + component classes
│       └── vite.config.js (proxy)
```

Data flow: Page → Redux slice → `services/api.js` (axios + JWT header) → Express route → controller → Mongoose model → MongoDB → JSON back → slice state → Recharts/UI.

6. Features

6.1 Core (Original) Features – all retained

#	Feature	Notes
1	Register / Login	JWT, bcrypt, profile update
2	Dashboard	Balance, month income/expense, donut, budget bars, recent tx
3	Transactions CRUD	Income + expense, filters, search, pagination
4	Categories CRUD	Custom categories with icons/emoji, income+expense types
5	Monthly Budgets	Per-category limits, progress bars, over-budget status
6	Savings Goals	Target vs current, contributions, progress
7	Recurring Expenses	Monthly frequency, active toggle, auto-posting when due
8	Analytics	Bar/line/pie charts: monthly trend, category, payment method

6.2 New & Enhanced Features (added in this rebuild)

#	Feature	What it does
1	Light / Dark mode	Class-based, persisted, one-click toggle
2	8 selectable themes	Profile → Appearance; one file per theme
3	Monthly Reports page	Month navigator, MoM comparison, insight banner ("spent 48% less than last month"), category table with change %, payment-method breakdown
4	CSV export	<code>GET /api/transactions/csv</code> streams a downloadable CSV of filtered data
5	Favorites	Star frequent transactions; dedicated favorites list
6	Receipt upload	Attach ≤2 MB image per transaction; lightbox preview
7	Smart notifications	Budget-near/over, goal milestones, recurring-due, welcome – with de-duplication (<code>key</code> field)
8	Recurring auto-post	On login/app load, due recurring items are posted as real transactions
9	Multi-chart analytics	Vertical selector: Bar / Line / Pie / Area / Donut for each analysis
10	Tags	Free-form tags (<code>home</code> , <code>work</code>) on transactions + filter
11	Advanced filters	Date range, category, type, payment method, amount range, favorites-only
12	Empty & loading states	Dedicated EmptyState art + skeleton loaders on every page
13	Amount sanitization	Client + server validation (rejects <code>abc</code> , <code>-5</code> , <code>>2</code> decimals)
14	Centralized env	One config file per side for all environment variables
15	Security pass	helmet, CORS allow-list, generic auth errors, ownership checks on every query
16	Micro-interactions	Row flash on create, hover lifts, modal animation, focus rings

6.3 Original vs Rebuild

Area	Original	Rebuild
State	local/ <code>useState</code> patterns	Redux Toolkit slices
Themes	fixed palette	8 themes + dark mode
Reports	basic totals	MoM insight + category change %
Export	none	CSV
Validation	client mostly	client and server (<code>toAmount</code>)
Notifications	basic	de-duplicated smart alerts
Config	scattered <code>process.env</code>	2 centralized config files
Design	generic	Option 8 + JetBrains Mono

7. Database Design (MongoDB Collections)

Model	Key fields
User	name, email (unique), password (bcrypt), currency, monthlyIncome, theme prefs
Category	user, name, type (income/expense), icon, color
Transaction	user, type, amount, category→Category, description, date, paymentMethod, <code>tags[]</code> , <code>receipt</code> , <code>favorite</code> , notes
Budget	user, category→Category, amount, month, year
SavingsGoal	user, title, targetAmount, currentAmount, targetDate, icon
RecurringExpense	user, title, amount, category, frequency, nextDate, paymentMethod, active
Notification	user, type, title, message, <code>key</code> (de-dup), read

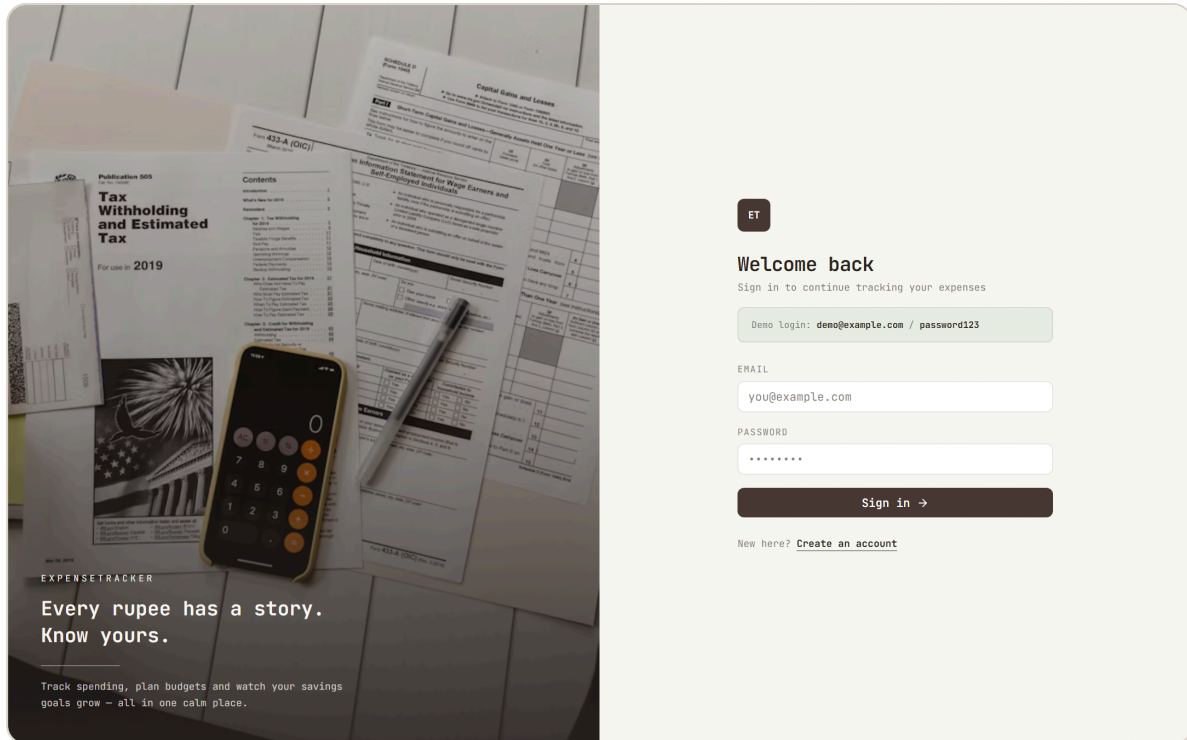
All finance models are indexed by `user` + `date` / `month-year` for fast aggregation. Every query filters by `user: req.user._id` – users can never see each other's data.

8. API Reference (40 endpoints)

Router	Endpoints
/api/auth	POST /register , POST /login , GET /me , PUT /profile
/api/transactions	GET / , POST / , GET /:id , PUT /:id , DELETE /:id , GET /csv , GET /favorites , PATCH /:id/favorite , PATCH /:id/receipt
/api/categories	GET / , POST / , PUT /:id , DELETE /:id (blocked if used by recurring)
/api/budgets	GET / , POST / , PUT /:id , DELETE /:id
/api/goals	GET / , POST / , PUT /:id , DELETE /:id , POST /:id/add-money
/api/recurring-expenses	GET / , POST / , PUT /:id , DELETE /:id , PATCH /:id/toggle
/api/analytics	GET /dashboard , GET /monthly , GET /categories , GET /payment-methods , GET /reports
/api/notifications	GET / , PATCH /read-all , PATCH /:id/read , DELETE /:id

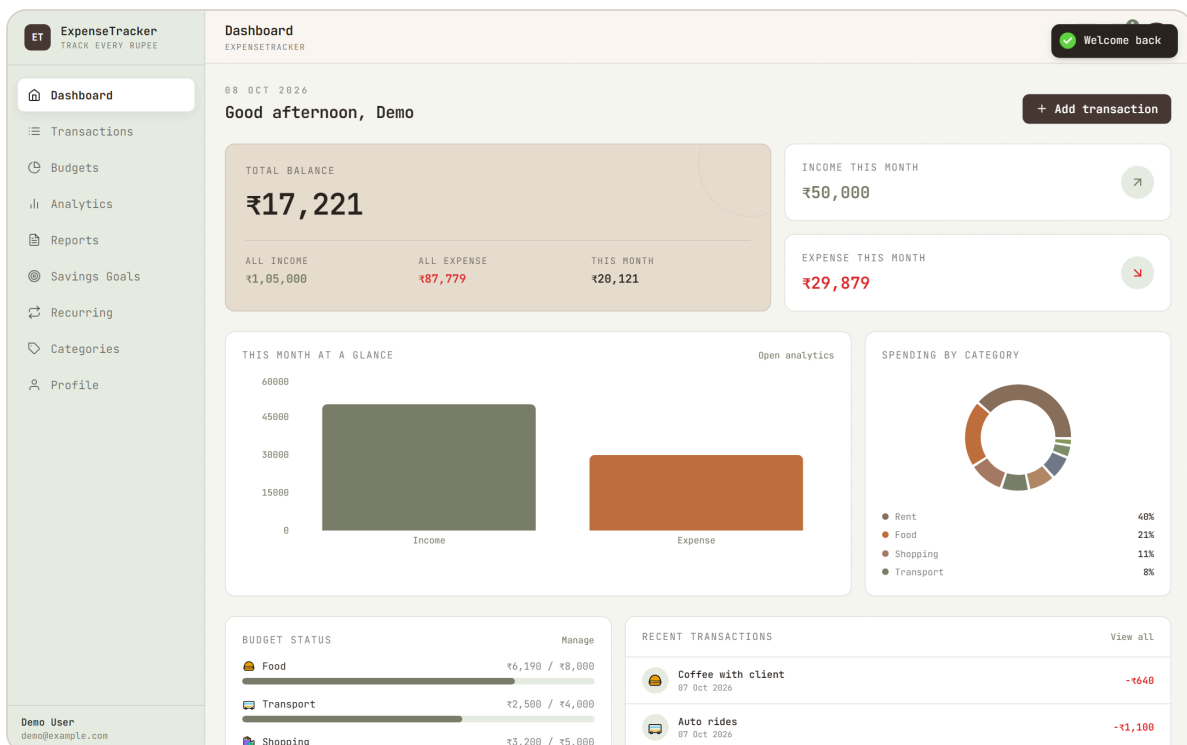
9. Key Screenshots

9.1 Login



Split-screen login with Unsplash imagery (with offline fallback) and monospace form.

9.2 Dashboard



Balance hero card, income/expense stats, month-at-a-glance bar chart, category donut, budget progress bars, recent transactions.

9.3 Transactions + Add Modal

ET ExpenseTracker
TRACK EVERY RUPEE

Dashboard

Transactions

Budgets

Analytics

Reports

Savings Goals

Recurring

Categories

Profile

Demo User
demo@example.com

Transactions
EXPENSETRACKER

32 RECORDS

All transactions

Export CSVAdd transaction

Search descriptions...FiltersFavorites

DATE	DESCRIPTION	CATEGORY	PAYMENT	AMOUNT	ACTIONS
07 Oct 2026	Coffee with client coffee work	Food	Cash	-₹640	⬆️ ⭐ ✎️ 🗑️
07 Oct 2026	Auto rides commute	Transport	Cash	-₹1,100	⬆️ ⭐ ✎️ 🗑️
06 Oct 2026	Gym supplements	Health	UPI	-₹990	⬆️ ⭐ ✎️ 🗑️
06 Oct 2026	Running shoes fitness	Shopping	Credit Card	-₹3,200	⬆️ ⭐ ✎️ 🗑️
05 Oct 2026	Team lunch work	Food	Credit Card	-₹1,750	⬆️ ⭐ ✎️ 🗑️
05 Oct 2026	Streaming + games	Entertainment	Credit Card	-₹2,400	⬆️ ⭐ ✎️ 🗑️
05 Oct 2026	Mobile Plan	Mobile	UPI	-₹499	⬆️ ⭐ ✎️ 🗑️
04 Oct 2026	Phone bill	Bills	Net Banking	-₹2,100	⬆️ ⭐ ✎️ 🗑️
03 Oct 2026	Metro card top-up commute	Transport	UPI	-₹1,400	⬆️ ⭐ ✎️ 🗑️
02 Oct 2026	Monthly groceries	Food	Debit Card	-₹3,800	⬆️ ⭐ ✎️ 🗑️

ET ExpenseTracker
TRACK EVERY RUPEE

Dashboard

Transactions

Budgets

Analytics

Reports

Savings Goals

Recurring

Categories

Profile

Demo User
demo@example.com

Transactions
EXPENSETRACKER

32 RECORDS

All transactions

Export CSVAdd transaction

Search descriptions...FiltersFavorites

ADD TRANSACTION

ExpenseIncome

AMOUNTDATE

010/08/2026

CATEGORY

Select category

DESCRIPTION

What was it for?

PAYMENT METHODTAGS (COMMA SEPARATED)

Cashhome, work

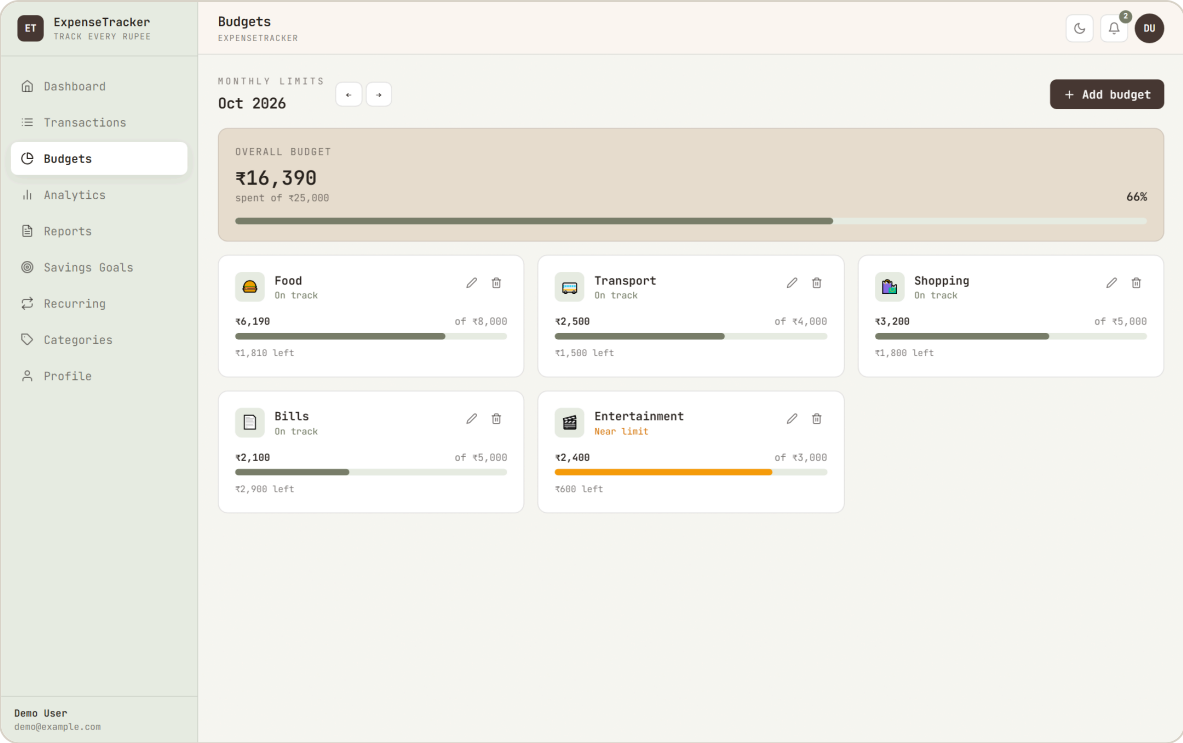
NOTES

Optional notes

CancelAdd transaction

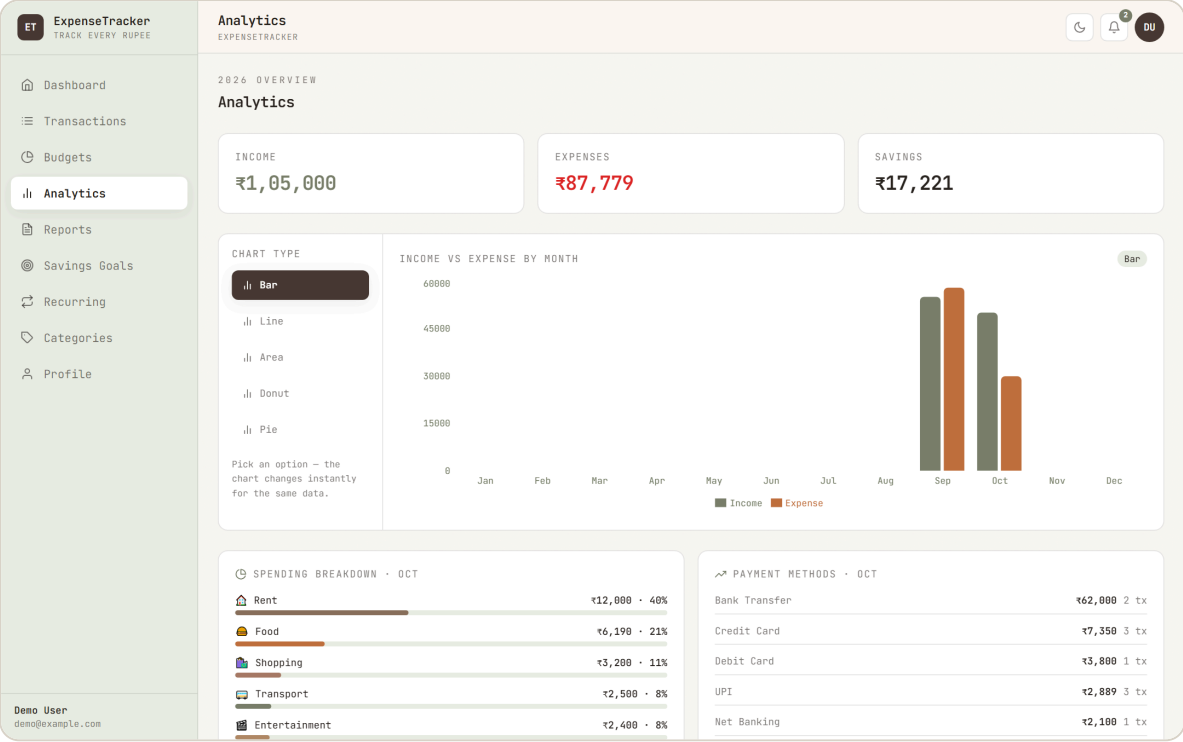
Filters (search, type, category, date range, tags), CSV export, favorites; sanitized amount input in the modal.

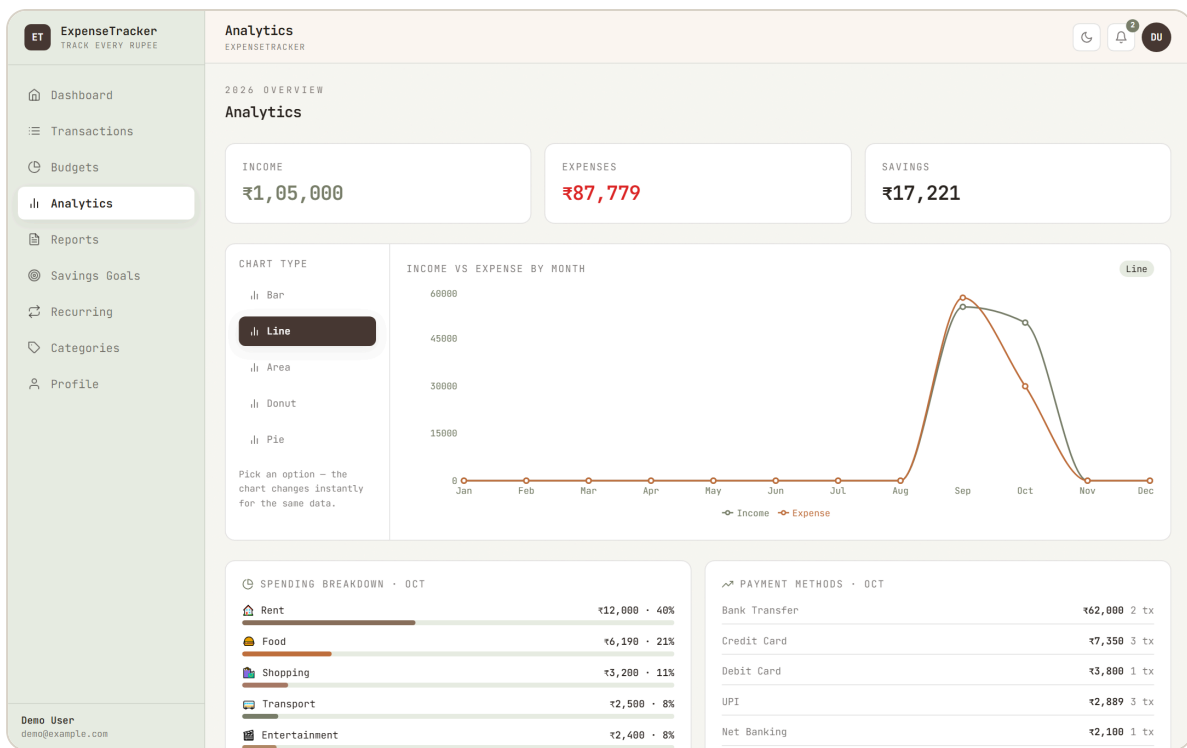
9.4 Budgets



Per-category monthly limits with progress bars and over-budget highlighting.

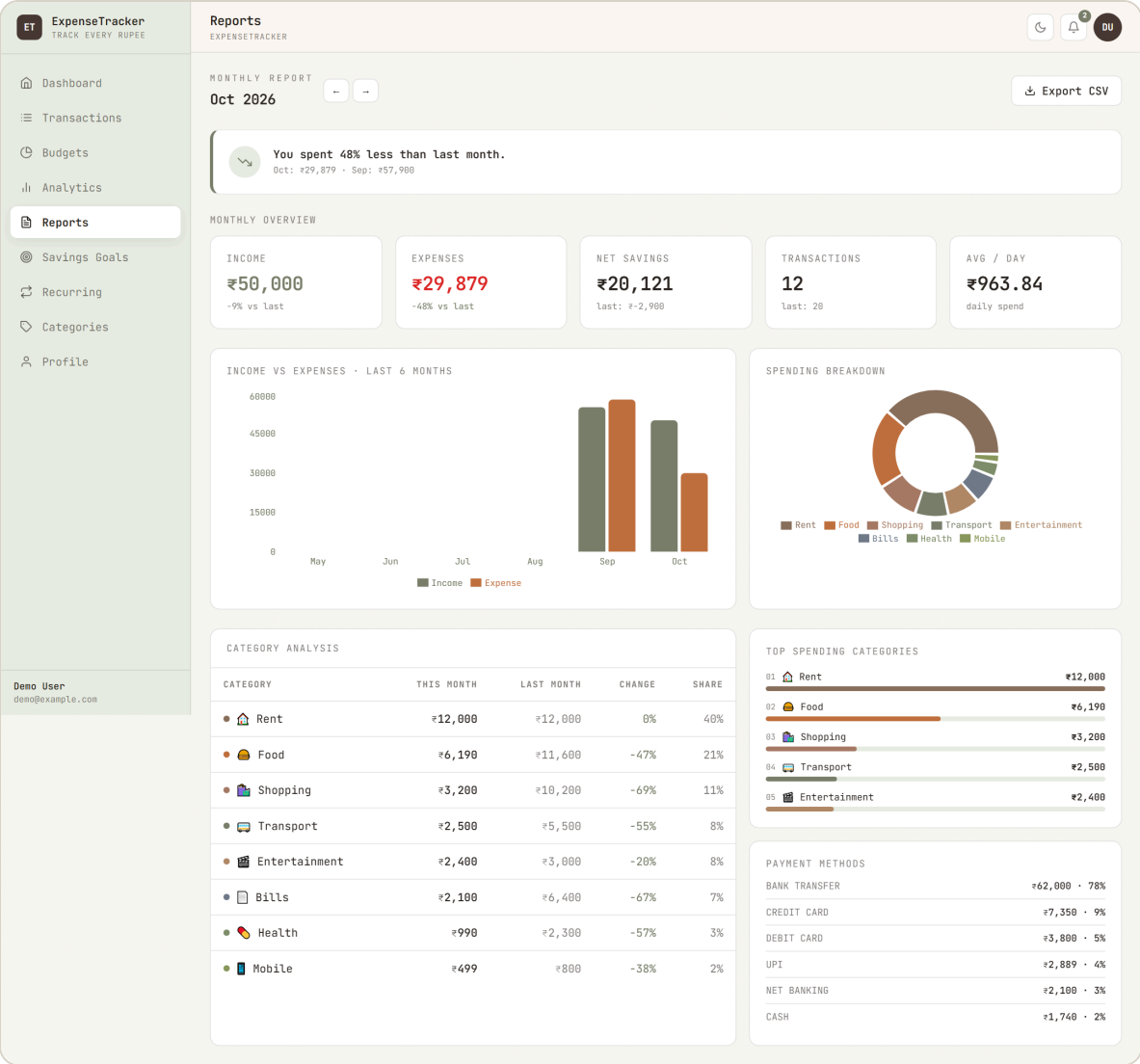
9.5 Analytics (chart switching)





Vertical chart-type selector switches Bar → Line (Pie/Area/Donut also available) without leaving the page.

9.6 Monthly Report



9.7 Goals / Recurring / Categories

ETExpenseTracker
TRACK EVERY RUPEE

Dashboard

Transactions

Budgets

Analytics

Reports

Savings Goals

Recurring

Categories

Profile

Demo User
demo@example.com

Savings Goals
EXPENSETRACKER

SAVE FOR WHAT MATTERS

Savings goals

+ New goal

Emergency Fund
by 31 Dec 2027

₹62,000of ₹1,00,000

62% complete

Add money

Goa Trip
by 15 Jan 2027

₹18,000of ₹30,000

60% complete

Add money

New Laptop
by 31 Mar 2027

₹45,000of ₹80,000

56% complete

Add money

ETExpenseTracker
TRACK EVERY RUPEE

Dashboard

Transactions

Budgets

Analytics

Reports

Savings Goals

Recurring

Categories

Profile

Demo User
demo@example.com

Recurring Expenses
EXPENSETRACKER

AUTO-POSTED WHEN DUE

Recurring expenses

+ Add recurring

Due expenses are posted to your transactions automatically whenever you open the dashboard.

ACTIVE (5)

Netflix
Monthly · next 01 Nov 2026

₹649UPI

Active

Apartment Rent
Monthly · next 01 Nov 2026

₹12,000UPI

Active

Gym Membership
Monthly · next 03 Nov 2026

₹1,500UPI

Active

Internet
Monthly · next 05 Nov 2026

₹1,499UPI

Active

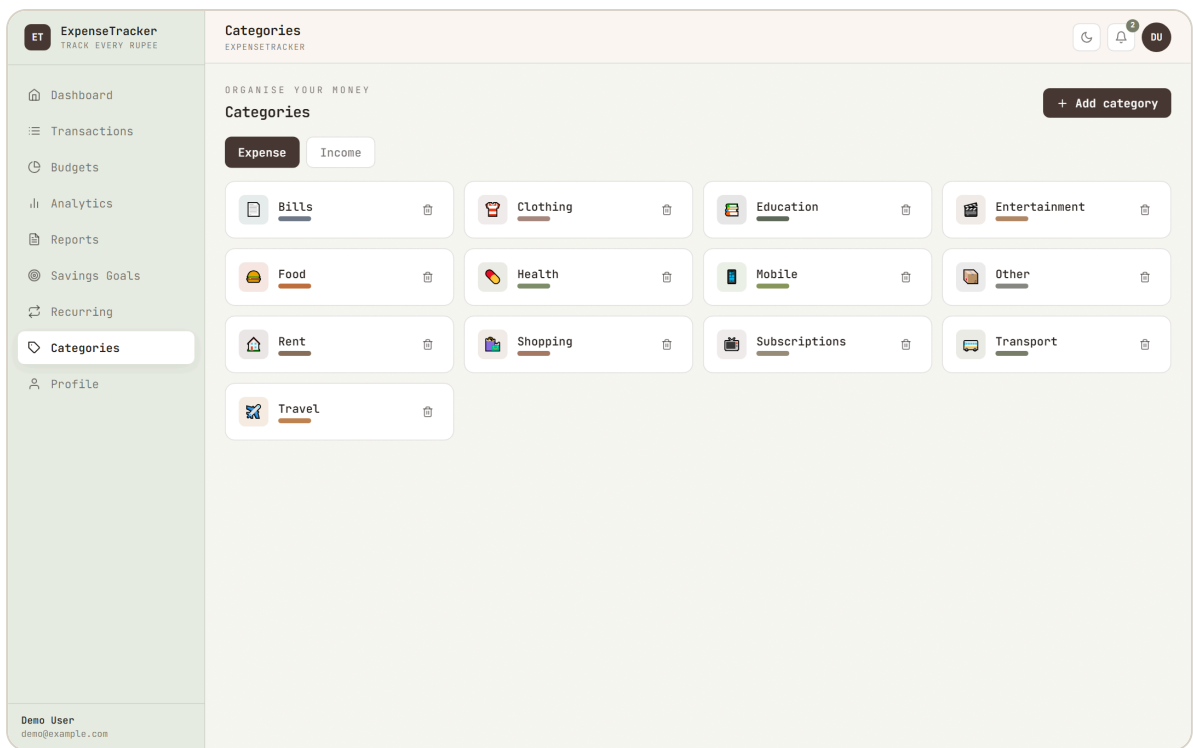
Mobile Plan
Monthly · next 05 Nov 2026

₹499UPI

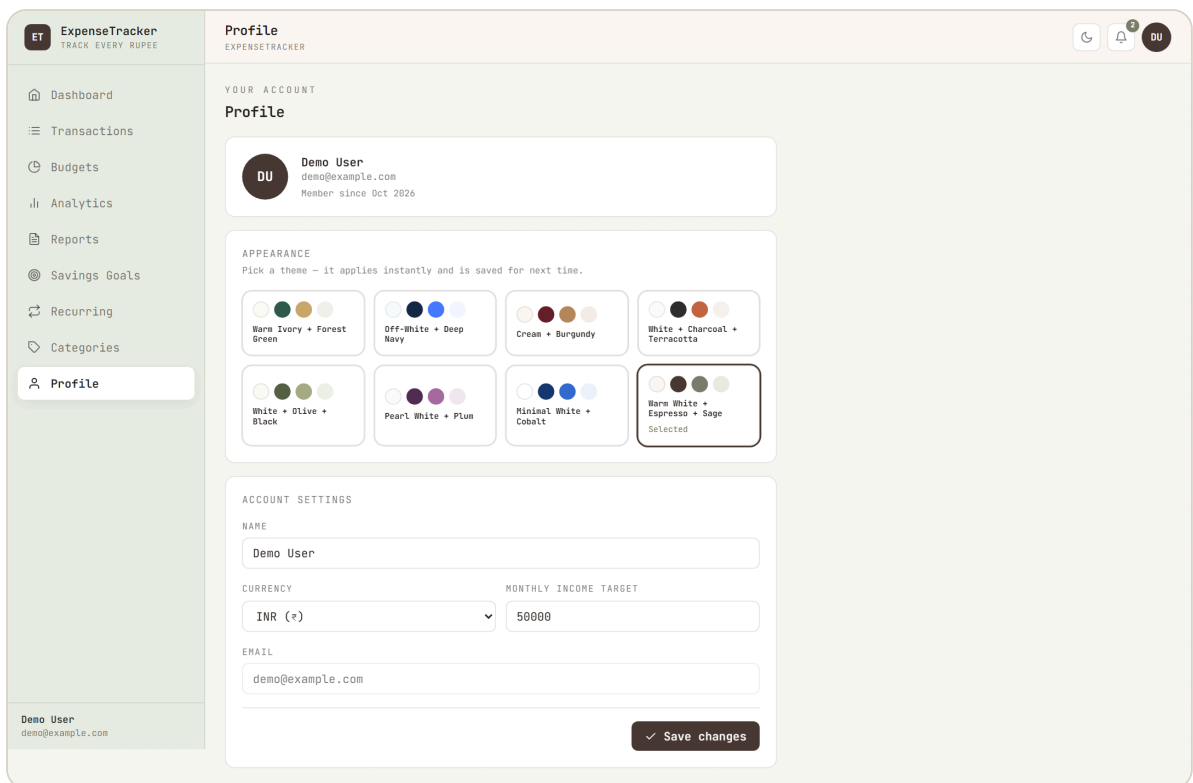
Active

ExpenseTracker — Final Project Report

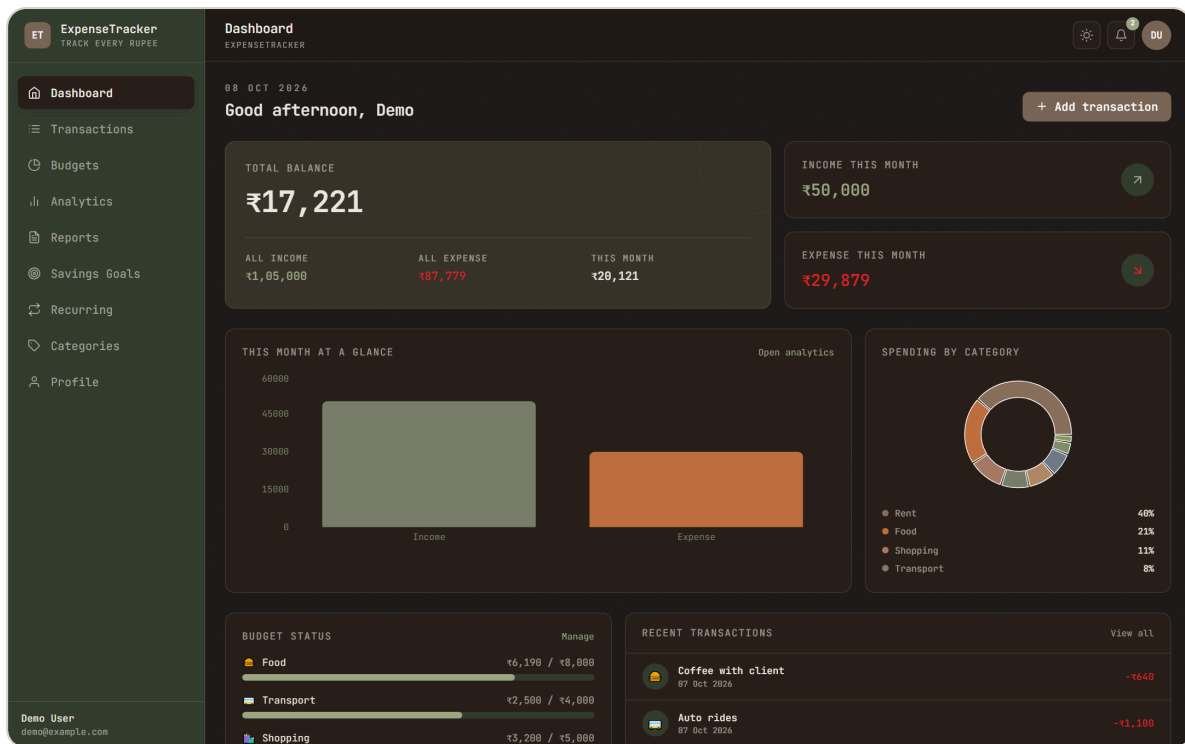
Page 15 of 29



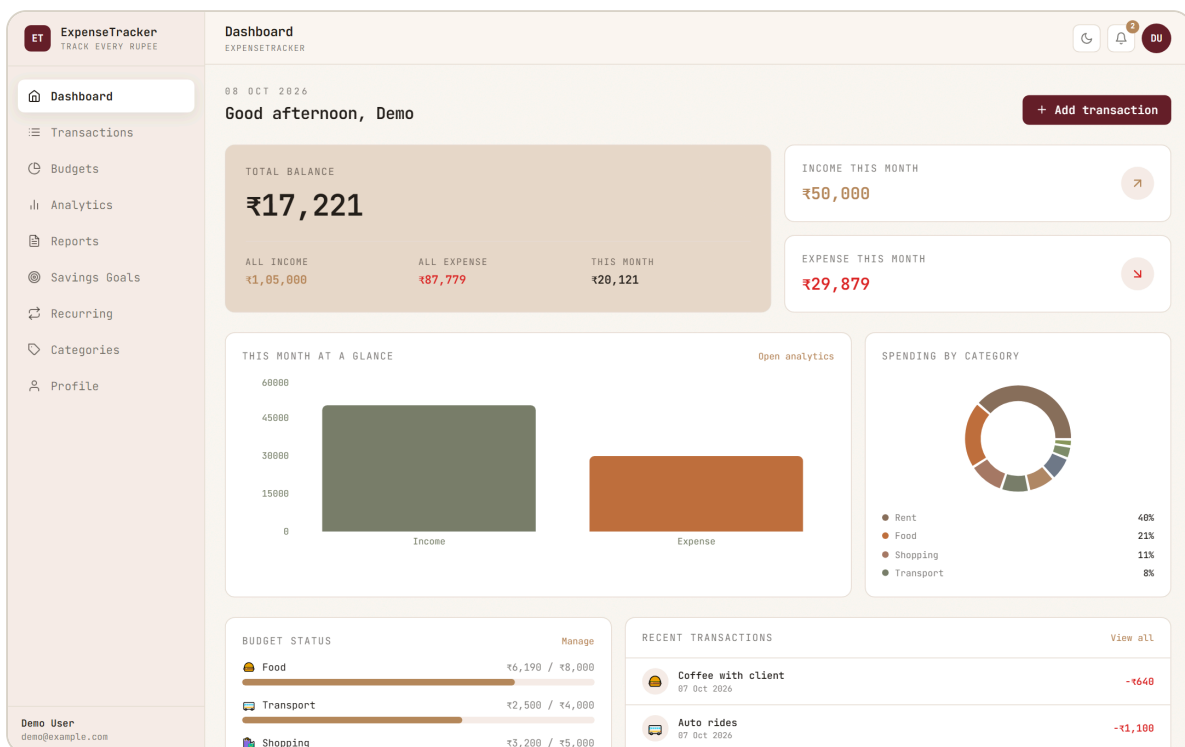
9.8 Theme Selector & Dark Mode



Profile → Appearance: 8 theme cards; current theme highlighted.

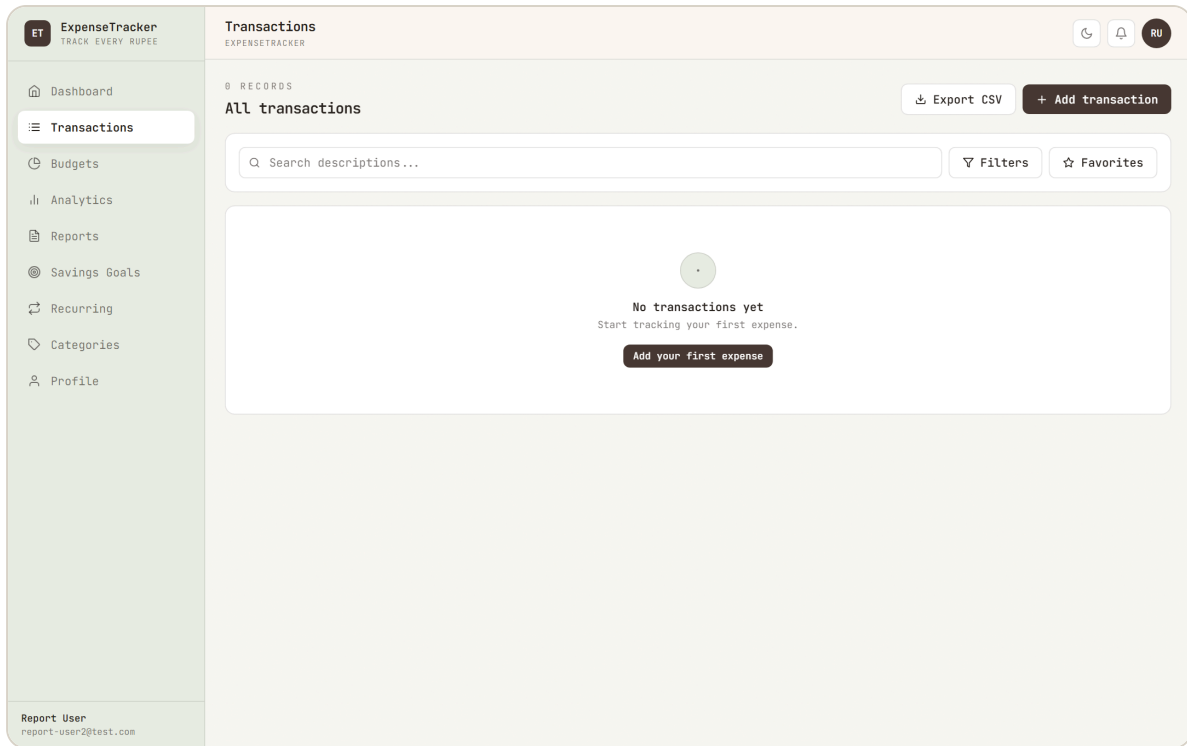


Dark mode dashboard – same data, inverted palette.



Same dashboard after one click on "Cream + Burgundy" – full re-skin via CSS variables.

9.9 Empty State



Fresh account – friendly empty state instead of a blank table.

10. Code Highlights

10.1 Dashboard aggregation (sequential, easy to explain)

```
await checkBudgets(userId, month, year, budgetStatus);

res.json({
  success: true,
  data: {
    balance: allTime.income - allTime.expense,
    totalIncome: allTime.income,
    totalExpense: allTime.expense,
    monthlyIncome: monthTotals.income,
    monthlyExpense: monthTotals.expense,
    monthlySavings: monthTotals.income - monthTotals.expense,
    categoryBreakdown,
    recentTransactions,
    budgetStatus,
  },
});
} catch (error) {
  next(error);
}
};

exports.getMonthly = async (req, res, next) => {
  try {
    const year = parseInt(req.query.year) || new Date().getFullYear();
    const startDate = new Date(year, 0, 1);
    const endDate = new Date(year, 11, 31, 23, 59, 59, 999);

    const rows = await Transaction.aggregate([
      { $match: { user: req.user._id, date: { $gte: startDate, $lte: endDate } } },
      { $group: { _id: { month: { $date: '$date' }, type: '$type' }, total: { $sum: '$amount' } } },
    ]);

    const result = [];
    for (let m = 1; m <= 12; m++) {
      const monthRow = { month: m, name: monthNames[m - 1], income: 0, expense: 0, savings: 0 };
      for (const row of rows) {
        if (row._id.month !== m) continue;
        if (row._id.type === 'income') monthRow.income = row.total;
        if (row._id.type === 'expense') monthRow.expense = row.total;
      }
      monthRow.savings = monthRow.income - monthRow.expense;
      result.push(monthRow);
    }

    res.json({ success: true, data: result });
  } catch (error) {
    next(error);
  }
};

exports.getCategoriesAnalytics = async (req, res, next) => {
  try {
    const now = new Date();
    const month = parseInt(req.query.month) || now.getMonth() + 1;
    const year = parseInt(req.query.year) || now.getFullYear();
```

server/src/controllers/analyticsController.js – Dashboard aggregation (sequential awaits, no Promise.all)

Note the fixed MongoDB pitfall: non-`_id` fields inside `$group` must use accumulators (`{ $first: '$field' }`), not bare paths – this exact bug was found and fixed during testing.

10.2 Recurring auto-posting

```
const today = new Date();
today.setHours(0, 0, 0, 0);

const dueItems = await RecurringExpense.find({
  user: userId,
  active: true,
  nextDate: { $lte: today },
});

for (const item of dueItems) {
  await Transaction.create({
    user: userId,
    type: 'expense',
    amount: item.amount,
    category: item.category,
    description: item.title,
    date: item.nextDate,
    paymentMethod: item.paymentMethod,
    isRecurring: true,
    recurringExpense: item._id,
  });

  item.nextDate = nextDateAfter(item.nextDate, item.frequency);
  await item.save();

  await recurringPosted(userId, item.title, item.amount);
}
```

server/src/utils/processRecurring.js - Recurring expense auto-posting

Due recurring items become real transactions automatically.

10.3 Smart notifications

```
const Notification = require('../models/Notification');

const createOnce = async (userId, type, title, message, key) => {
  const alreadySent = await Notification.findOne({ user: userId, key });
  if (alreadySent) return;

  await Notification.create({ user: userId, type, title, message, key });
};

const checkBudgets = async (userId, month, year, budgets) => {
  for (const item of budgets) {
    const categoryName = item.category ? item.category.name : 'Budget';

    if (item.percentage >= 100) {
      await createOnce(
        userId,
        'budget_exceeded',
        'Budget exceeded',
        `${categoryName} budget is over by ${Math.round(item.percentage - 100)}%`,
        `budget:${year}:${month}:${categoryName}:over`
      );
    } else if (item.percentage >= 80) {
      await createOnce(
        userId,
        'budget_warning',
        'Budget warning',
        `${categoryName} budget is ${Math.round(item.percentage)}% used`,
        `budget:${year}:${month}:${categoryName}:warn`
      );
    }
  }
}
```

server/src/utils/notify.js - Smart notification helper with de-duplication

De-duplication via a stable `key` so the same budget warning never spams the bell.

10.4 One theme = one file

```
const theme = {
  id: 'option8',
  name: 'Option 8 - Warm White + Espresso + Sage',
  light: {
    bg: '#FAF8F3',
    text: '#29231F',
    card: '#FFFFFF',
    border: '#29231F1F',
    primary: '#493B32',
    primaryText: '#FFFFFF',
    primaryHover: '#3A2F27',
    accent: '#78816A',
    sage: '#E9EDE4',
    beige: '#E9DFD1',
  },
  dark: {
    bg: '#1E1A16',
    text: '#EFE9E1',
    card: '#28221C',
    border: '#EFE9E11F',
    primary: '#7A6656',
    primaryText: '#FFFFFF',
    primaryHover: '#8C7767',
    accent: '#9BA586',
    sage: '#333D2F',
    beige: '#3A322A',
  },
};

export default theme;
```

client/src/themes/option8WarmEspressoSage.js - One theme = one separate file

Adding a 9th theme means adding one file + one registry entry – nothing else changes.

10.5 Chart selector

```
</div>

<div className="grid gap-4 lg:grid-cols-2">
  <div className="card p-5">
    <div className="mb-4 flex items-center gap-2">
      <FiPieChart size={14} className="text-accent" />
      <p className="section-title">Spending breakdown · {monthName(now.getMonth() + 1)}</p>
    </div>
    {breakdown.length === 0 ? (
      <EmptyState title="No spending this month" message="Expenses will appear here once you add them." />
    ) : (
      <div className="space-y-3">
        {breakdown.map((item) => (
          <div key={item._id}>
            <div className="mb-1 flex items-center justify-between text-xs">
              <span className="flex items-center gap-2">
                <span>{item.icon}</span> {item.name}
              </span>
              <span className="font-medium">
                {formatCurrency(item.total, currency)} · {Math.round(item.percentage)}%
              </span>
            </div>
            <div className="h-1.5 w-full overflow-hidden rounded-full bg-sage">
              <div
                className="h-full rounded-full transition-all duration-300"
                style={{ width: `${item.percentage}%`, backgroundColor: item.color }}
              />
            </div>
          </div>
        ))}
      </div>
    )}
  </div>

  <div className="card p-5">
    <div className="mb-4 flex items-center gap-2">
```

client/src/pages/Analytics.jsx – Vertical multi-chart selector

Data stays mounted; only the chart component swaps – animation stays smooth.

10.6 Amount sanitization

```
export const sanitizeAmountInput = (value) => {
  let cleaned = String(value).replace(/^[^0-9.]/g, '');

  const parts = cleaned.split('.');
  if (parts.length > 2) {
    cleaned = parts[0] + '.' + parts.slice(1).join('');
  }

  const [whole, decimal] = cleaned.split('.');
  if (decimal !== undefined) {
    cleaned = whole + '.' + decimal.slice(0, 2);
  }

  return cleaned;
};

export const isValidAmount = (value) => {
  const number = Number(value);
  return value !== '' && Number.isFinite(number) && number > 0;
};

export const amountErrorMessage = (value) => {
  if (value === '' || value === null || value === undefined) return 'Amount is required';
  const number = Number(value);
  if (!Number.isFinite(number)) return 'Please enter a valid number';
  if (number <= 0) return 'Amount must be greater than 0';
  return '';
};
```

client/src/utils/amount.js – Amount sanitization helper

Client blocks bad keystrokes; server independently rejects `abc`, `-5`, `12.345` (→ 400).

10.7 CSV export

```
if (!transaction) {
  return res.status(404).json({ success: false, message: 'Transaction not found' });
}

if (req.body.amount !== undefined) {
  const validAmount = toAmount(req.body.amount);
  if (validAmount === null) {
    return res.status(400).json({ success: false, message: 'Please provide a valid amount greater than 0' });
  }
  transaction.amount = validAmount;
}

const allowedFields = ['type', 'category', 'description', 'date', 'paymentMethod', 'tags', 'notes', 'receipt', 'isPa
for (const field of allowedFields) {
  if (req.body[field] !== undefined) {
    transaction[field] = req.body[field];
  }
}

await transaction.save();
await transaction.populate('category', 'name icon color type');
await alertBudgetsFor(req.user, new Date(transaction.date));

res.json({ success: true, data: transaction });
} catch (error) {
  next(error);
}
};

exports.deleteTransaction = async (req, res, next) => {
  try {
    const transaction = await Transaction.findOneAndDelete({ _id: req.params.id, user: req.user.id });

    if (!transaction) {
      return res.status(404).json({ success: false, message: 'Transaction not found' });
    }

    removeReceiptFile(transaction.receipt);
  }
};
```

server/src/controllers/transactionController.js – CSV export endpoint

Streams a proper `text/csv` download with date-stamped filename.

11. Security & Validation

1. Passwords hashed with bcrypt (10 rounds); never returned in APIs.
2. JWT (expiry configured) in `Authorization: Bearer` header; axios interceptor attaches/ clears it.
3. Ownership checks – every find/update/delete includes `user: req.user._id`.
4. Server-side validation with a `sanitize.js` helper – invalid amounts, empty titles, bad enums → 400 with clear messages.
5. helmet security headers + CORS allow-list.
6. Multer limits: 2 MB, image mime-types only.
7. Env centralization – only `server/src/config/env.js` and `client/src/config/env.js` read raw env vars; secrets live in `.env` (git-

ignored, `.env.example` provided).

8. **No third-party APIs** – everything runs locally; auth pages use static Unsplash images with graceful fallback.

12. How to Run

Prerequisites: Node.js $\geq$ 18, MongoDB running locally (default port 27017).

```
# 1. Backend
cd server
npm install
cp .env.example .env      # or create .env manually
npm run seed              # creates demo + saurabh users with data
npm run dev               # http://localhost:5000

# 2. Frontend (new terminal)
cd client
npm install
npm run dev               # http://localhost:5173
```

Login accounts (seeded):

Email	Password	Data
demo@example.com	password123	31 transactions, 5 budgets, 5 recurring, 3 goals
saurabh@gmail.com	saurabh123	18 transactions, 3 budgets, 3 recurring, 3 goals

13. Testing & Verification

Area	Test	Result
Auth	register / login / getMe	✓ 200
Dashboard	summary aggregation	✓ 200 (after \$group fix)
Reports	MoM + categories + payment methods	✓ 200
CRUD	transactions, budgets, goals, recurring, categories	✓ 200
CSV	GET /transactions/csv	✓ valid CSV stream
Validation	amount abc , -5	✓ 400 rejected
Validation	amount 123.456	✓ stored 123.46
Validation	budget 0	✓ 400 rejected
Category delete	delete category used by recurring	✓ blocked
Frontend	npm run build	✓ success (948 modules)
Seeding	both demo users created	✓ verified

Known cosmetic note: client build prints a chunk-size warning (>500 kB) because Recharts bundles all chart types – a warning only, not an error; handled by lazy-loading charts in future scope.

14. Challenges Faced & Solutions

Challenge	Solution
MongoDB error: The field 'name' must be an accumulator object	Non- <code>_id</code> fields in <code>\$group</code> need accumulators; replaced bare paths with <code>{ \$first: '\$name' }</code> in 3 pipelines
Tailwind <code>.table-row { display: table-row }</code> clashed with the custom <code>.table-row</code> component class (broke a flex row)	Renamed component class to <code>.row-hover</code> ; zero <code>table-row</code> in built CSS now
Amount fields accepted <code>abc</code> / negative / <code>>2</code> decimals	Added <code>sanitizeAmountInput</code> (client) + <code>toAmount</code> (server), both unit-tested with curl
Port conflicts (<code>EADDRINUSE</code>) during restarts	Single dev process per port; ports centralized in <code>.env</code>
Scattered <code>process.env</code> / <code>import.meta.env</code>	Restricted all raw env access to exactly two config files

15. Future Scope

Automation & Scheduling

1. **Cron scheduler** (node-cron) – post recurring expenses even when the user never opens the app (currently posts on app load).
2. **Scheduled report emails** – monthly PDF report mailed on the 1st.
3. **Budget breach push alerts** – browser Web Push / email when spending crosses 80% / 100%.

Intelligence

4. **AI category suggestions** – auto-tag transactions from description ("Swiggy" → Food).
5. **Anomaly detection** – flag unusually large or unusual-category spending.
6. **Forecasting** – predict month-end balance from current run-rate.
7. **Natural-language add** – "coffee 120 yesterday" → parsed transaction.

Data Ingestion

8. **Bank statement import** – CSV/OFX upload with column mapping.
9. **Receipt OCR** – extract merchant/amount/date from uploaded receipts.
10. **UPI/bank API integration** (via account aggregator consent) for auto-sync.

Product 11. **Multi-currency** with daily exchange rates + trip-wise grouping. 12. **Shared/family budgets** – household accounts with roles. 13. **PWA + offline mode** – installable app, queue entries offline, sync later. 14. **React Native mobile app** – reuse the REST API as-is. 15. **Gamification** – streaks, savings badges, monthly challenges. 16. **Custom dashboards** – drag-and-drop widgets, user-chosen metrics. 17. **Charts** → **image/PDF export** for presentations. 18. **i18n** – Hindi/regional language UI.

Engineering 19. **Refresh-token rotation** + **logout-all devices**. 20. **Rate limiting & brute-force lockout** (express-rate-limit). 21. **Automated tests** – Jest + Supertest API suite, React Testing Library for components. 22. **CI/CD** – GitHub Actions lint/test/build; Dockerfile for one-command deploys. 23. **Code splitting** per route to silence the Recharts bundle warning. 24. **Structured logging & monitoring** (pino + error tracker). 25. **Audit log** – history of edits/deletes for every transaction.

16. Conclusion

ExpenseTracker demonstrates an end-to-end MERN application with a polished, distinctive UI (Option 8 palette + JetBrains Mono), a full personal-finance feature set, and a set of enhancements – themes, dark mode, CSV export, receipts, smart notifications, richer reports – that take it beyond basic CRUD. The codebase is intentionally structured in small, single-purpose files with sequential, readable control flow, making it straightforward to demonstrate and defend in a project viva. Validation runs on both client and server, data is strictly per-user, and the whole system runs locally with two commands and no external services.

Appendix A – Environment Variables

server/.env

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/expenseTracker
JWT_SECRET=<your-secret>
JWT_EXPIRES_IN=7d
CLIENT_URL=http://localhost:5173
```

client/.env

```
VITE_API_BASE_URL=/api
VITE_APP_NAME=ExpenseTracker
VITE_APP_TAGLINE=Track every rupee
VITE_REQUEST_TIMEOUT=15000
```

Appendix B – Seed Data

User	Email	Password	Contents
Demo User	demo@example.com	password123	31 transactions (Sep + Oct 2026), 5 budgets, 5 recurring, 3 goals
Saurabh	saurabh@gmail.com	saurabh123	18 transactions (Sep + Oct 2026), 3 budgets, 3 recurring, 3 goals